

Laboratory 07

Stack Overflow Detection & Memory Analysis

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXXN236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Estimated time	2–3 hours
CLO mapping	CLO 4 & CLO 5

1. Objectives

By completing this lab you will be able to:

1. **Induce** a stack overflow in a FreeRTOS task using a recursive function.
2. **Detect** the overflow using `configCHECK_FOR_STACK_OVERFLOW = 2` before corruption spreads.
3. **Right-size** task stacks using `uxTaskGetStackHighWaterMark`.
4. **Analyse** total heap usage with `xPortGetFreeHeapSize` and the linker map file.
5. **Convert** one task to static allocation and verify heap usage drops.

2. Prerequisites

- ☐ Lab 03 producer-consumer system (used as the baseline in Part C).
- ☐ ARM GNU Toolchain and pyocd working.
- ☐ A text editor with hex search (for observing the 0xA5 pattern).

3. Background

3.1. Stack Overflow Consequences

Every FreeRTOS task has its own stack allocated from the heap. If a task's stack grows beyond its allocation, it silently overwrites the adjacent heap block — which may contain another task's TCB, stack, or queue buffer. The crash typically manifests as a hard fault far from the actual cause.

FreeRTOS provides two overflow detection mechanisms:

Method	Config value	How it works	Catches
1	1	Check <code>SP < stack base</code> at each context switch	Overflows that leave <code>SP</code> below base
2	2	Fill entire stack with 0xA5 at creation; check last 20 bytes at each switch	Most overflows, including those that recover

4. Project Structure

```
lab07_stack_overflow/
  Makefile
  linker_MCXXN236.ld
  src/
    main.c           <- task creation
    overflow_demo.c/.h <- Part A: intentional overflow
```

```

stack_sizing.c/.h      <- Part B: sizing the Lab 03 pipeline
static_alloc.c/.h     <- Part C: static allocation
FreeRTOSConfig.h
uart.h / uart.c
FreeRTOS-Kernel/

```

5. Part A — Induce and Detect a Stack Overflow

5.1. Step 1 — Create a Task with a Small Stack

```

1  /* overflow_demo.c */
2
3  /* Each call adds ~64 bytes to the stack */
4  static uint32_t recursive_sum(uint32_t n)
5  {
6      uint32_t local_buffer[8];
7      for (int i = 0; i < 8; i++) local_buffer[i] = i + n;
8      if (n == 0) return 0;
9      return local_buffer[0] + recursive_sum(n - 1);
10 }
11
12 void vOverflowTask(void *pv)
13 {
14     for (;;) {
15         uart_printf("[OVF] starting recursive_sum(60)...\r\n");
16         uint32_t result = recursive_sum(60);
17         uart_printf("[OVF] result = %lu (you should never see this)\r\n", result);
18         vTaskDelay(pdMS_TO_TICKS(2000));
19     }
20 }

```

In `main.c`, create the task with a very small stack (64 words = 256 bytes):

```

1  xTaskCreate(vOverflowTask, "Overflow", 64, NULL, 2, NULL);

```

5.2. Step 2 — First Run: No Detection

Build and flash with detection **disabled**:

```

1  /* FreeRTOSConfig.h */
2  #define configCHECK_FOR_STACK_OVERFLOW 0

```

Observe the terminal. The overflow will likely cause a HardFault or garbled output, but no useful diagnostic message.

✓ Checkpoint A1

UART capture showing the crash — note that the crash is non-deterministic and may look different each run.

5.3. Step 3 — Enable Overflow Detection

```

1  /* FreeRTOSConfig.h */
2  #define configCHECK_FOR_STACK_OVERFLOW 2

```

Implement the hook in `overflow_demo.c`:

```

1 void vApplicationStackOverflowHook(TaskHandle_t xTask, char *
   pcTaskName)
2 {
3     taskDISABLE_INTERRUPTS();
4     uart_printf("\r\n!!! STACK OVERFLOW in task: %s !!!\r\n",
       pcTaskName);
5     for (;;) {} /* halt so we can read the message */
6 }

```

Rebuild and flash.

✓ Checkpoint A2

UART capture showing !!! STACK OVERFLOW in task: Overflow !!!

? Question A1

With `configCHECK_FOR_STACK_OVERFLOW = 2`, the check only fires at context switch time — not at the exact moment of overflow. Is there any scenario where the overflow happens, the stack “heals” before the switch, and the hook does NOT fire? When would you use Method 1 vs. Method 2?

? Question A2

The overflow hook receives `pcTaskName`. Can you always trust this name to identify the crashed task? (Hint: what was overwritten just before the hook was called?)

6. Part B — Right-Sizing Stack Depths

6.1. Step 1 — Find the High-Water Mark

Increase the overflow task’s stack to 512 words so it can run safely:

```

1 xTaskCreate(vOverflowTask, "Overflow", 512, NULL, 2, NULL);

```

After at least 30 seconds, read the high-water mark:

```

1 void vStackReportTask(void *pv)
2 {
3     vTaskDelay(pdMS_TO_TICKS(30000));
4     uart_printf("\r\n=== Stack High-Water Mark Report ===\r\n");
5     TaskHandle_t handles[] = { xOverflowHandle, xSensor1Handle,
   xLoggerHandle };
6     const char *names[] = { "Overflow", "Sensor1", "Logger" };
7     for (int i = 0; i < 3; i++) {
8         UBaseType_t hwm = uxTaskGetStackHighWaterMark(handles[i]);
9         uart_printf(" %-16s HWM = %u words remaining\r\n", names[i],
   hwm);
10    }
11    uart_printf("Free heap: %u bytes\r\n", xPortGetFreeHeapSize());
12    vTaskDelete(NULL);
13 }

```

6.2. Step 2 — Compute the Right-Sized Stack

For each task, apply the formula:

$$\text{right_sized} = (\text{allocated} - \text{HWM}) + \lceil (\text{allocated} - \text{HWM}) \times 0.20 \rceil$$

6.3. Step 3 — Apply to the Lab 03 Pipeline

Task	Alloc'd (words)	HWM remaining	Min used	Right-sized (+20%)
vSensor1Task				
vSensor2Task				
vLoggerTask				
vMonitorTask				
vOverflowTask				

✓ Checkpoint B

Filled-in high-water-mark table from UART output.

? Question B1

`uxTaskGetStackHighWaterMark` returns 4 for one of your tasks. What does this mean? Is this task safe to leave running? What is your next step?

? Question B2

The formula adds 20% headroom. What real-world factors could cause a task to use more stack in production than was observed in the lab? Name at least three.

7. Part C — Heap Analysis

7.1. Step 1 — Total Heap Usage

```

1 /* In main.c, before vTaskStartScheduler: */
2 uart_printf("Heap free before scheduler: %u bytes\r\n",
3             xPortGetFreeHeapSize());
4 uart_printf("Heap minimum ever free:      %u bytes\r\n",
5             xPortGetMinimumEverFreeHeapSize());

```

7.2. Step 2 — Linker Map Analysis

Build with map output enabled:

```
make MAP=1
```

Open `lab07.map` and locate the memory layout. Fill in this table:

Section	Size (bytes)	Location	Contains
<code>.text</code>		Flash	Code, string literals, ISR vectors
<code>.rodata</code>		Flash	<code>const</code> data
<code>.data</code>		SRAM	Initialised globals
<code>.bss</code>		SRAM	Zero-init globals (incl. <code>ucHeap</code>)
Stack (MSP)		SRAM	Main stack (before scheduler)

✓ Checkpoint C

Filled-in linker map section size table.

8. Part D — Static Allocation

8.1. Step 1 — Convert One Task to Static Allocation

Convert vLoggerTask from dynamic to static allocation:

```

1  /* static_alloc.c */
2  static StaticTask_t xLoggerTaskBuffer;
3  static StackType_t xLoggerStack[256];    /* right-sized from Part B
      */
4
5  TaskHandle_t xLoggerHandle = xTaskCreateStatic(
6      vLoggerTask, "Logger", 256, NULL, 1,
7      xLoggerStack, &xLoggerTaskBuffer
8  );

```

Ensure FreeRTOSConfig.h has:

```

1  #define configSUPPORT_STATIC_ALLOCATION    1
2  #define configSUPPORT_DYNAMIC_ALLOCATION  1
3
4  void vApplicationGetIdleTaskMemory(StaticTask_t **
      ppxIdleTaskTCBBuffer,
5                                     StackType_t **
      ppxIdleTaskStackBuffer,
6                                     uint32_t *
      pulIdleTaskStackSize)
7  {
8      static StaticTask_t xIdleTaskTCB;
9      static StackType_t xIdleTaskStack[configMINIMAL_STACK_SIZE];
10     *ppxIdleTaskTCBBuffer = &xIdleTaskTCB;
11     *ppxIdleTaskStackBuffer = xIdleTaskStack;
12     *pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
13 }

```

8.2. Step 2 — Measure Heap Change

Run xPortGetFreeHeapSize() before and after switching to static allocation. Expected: free heap increases by approximately sizeof(StaticTask_t) + 256*4 bytes.

✓ Checkpoint D

UART output showing free heap before and after the conversion.

? Question D1

With configSUPPORT_STATIC_ALLOCATION = 1 and configSUPPORT_DYNAMIC_ALLOCATION = 0 (all static), what are the benefits for a safety-critical system? What does the linker map tell you that xPortGetFreeHeapSize() cannot?

? Question D2

vApplicationGetIdleTaskMemory must be implemented when static allocation is enabled. Why does the idle task require static allocation even if all your application tasks use dynamic allocation?

9. Deliverables

#	Item	Details
1	Part A1 UART capture	Crash without detection
2	Part A2 UART capture	Stack overflow hook message
3	Part B HWM table	All Lab 03 tasks + overflow task
4	Part C map table	Linker section sizes
5	Part D UART capture	Heap usage before and after static conversion
6	Written answers	Questions A1–A2, B1–B2, D1–D2

10. Key Concepts Demonstrated

Concept	Where you saw it
Stack overflow → silent corruption	Part A (no detection)
<code>vApplicationStackOverflowHook</code>	Part A (with detection)
High-water-mark measurement and right-sizing	Part B
Heap usage accounting	Part C
Linker map section analysis	Part C
Static task allocation (<code>xTaskCreateStatic</code>)	Part D

11. Reference

Resource	Relevant sections
Barry, <i>Mastering the FreeRTOS Kernel</i>	Ch. 3 (heap schemes)
FreeRTOS docs	<code>configCHECK_FOR_STACK_OVERFLOW</code> , <code>uxTaskGetStackHighWaterMark</code> , <code>xTaskCreateStatic</code>
NXP MCXN236 Reference Manual	§SRAM memory map